

2021-2022 学年期末考试真题中文解析

第 2 题 Prolog 程序设计

题目翻译

2. (8 分) Prolog 程序。

(1) Member(X,Y): 判断 X 是否是 Y 的一个成员。(4 分)

例子:

```
?- Member(a,[a,b,c,d])
```

```
Yes
```

请设计 Member(X,Y)。

(2) Append(X,Y,Z): 把列表 X 和 Y 拼接起来。(4 分)

例子:

```
?- Append([a,b,c],[1,2,3],L)
```

```
L=[a,b,c,1,2,3]
```

```
Yes
```

请设计 Append(X,Y,Z)。

对应知识点

这道题对应第七章 Prolog 程序设计。重点不是复杂算法，而是递归定义列表关系。

解析

(1) **Member** 要判断一个元素是不是列表成员，最自然的思路就是：

1. 如果它正好等于表头，那么成功；
2. 如果不在表头，就到表尾里继续找。

因此可以写成：

```
member(X,[X|_]).
```

```
member(X,[_|T]) :- member(X,T).
```

第一条规则表示：若列表第一个元素就是 X，那么 X 是这个列表的成员。

第二条规则表示：若第一个元素不是我们要找的，就递归到尾部 T 中继续查找。

这就是列表递归的典型写法。它利用了 Prolog 的模式匹配：

- [X|_] 表示表头是 X，尾部是什么不重要；
- [_|T] 表示跳过表头，只保留尾部。

(2) **Append** 列表拼接也有非常标准的递归结构：

1. 空表和任何表拼接，结果还是那个表；
2. 若第一个表不空，就把它的表头保留下来，再把它的表尾与第二个表继续拼接。

所以可以写成：

```
append([],Y,Y).  
append([H|T],Y,[H|Z]) :- append(T,Y,Z).
```

解释如下：

- `append([],Y,Y)`. 表示空表和 Y 拼接后就是 Y；
- 第二条规则表示，若第一个表是 `[H|T]`，那么拼接结果的第一个元素仍然是 H，其余部分就是 T 和 Y 的拼接结果。

这道题真正考的是：**能不能把列表问题拆成“表头 + 表尾”的递归形式。**

第 3 题 CLIPS 家谱规则

题目翻译

3. (9 分) 已知下面这些用于描述家谱事实的 `deftemplate`：

```
(deftemplate father-of (slot father) (slot child))  
(deftemplate mother-of (slot mother) (slot child))  
(deftemplate male (slot person))  
(deftemplate female (slot person))  
(deftemplate parent-of (slot parent) (slot child))  
(deftemplate brother-of (slot brother) (slot person))
```

请编写规则，推出以下关系：

- (1) Parent
- (2) Brother

对应知识点

这道题对应第八章 CLIPS。核心在于**从已有事实推出新事实**，也就是写出规则左部条件和右部动作。

解析

(1) Parent 父亲和孩子的关系、母亲和孩子的关系，都能推出“父母”和孩子的关系。因此至少要写两条规则：

```
(defrule infer-parent-from-father  
  (father-of (father ?f) (child ?c))  
  =>  
  (assert (parent-of (parent ?f) (child ?c))))  
  
(defrule infer-parent-from-mother  
  (mother-of (mother ?m) (child ?c))  
  =>  
  (assert (parent-of (parent ?m) (child ?c))))
```

这两条规则的逻辑非常直接：

- 如果某人是父亲，那么他也是父母之一；

- 如果某人是母亲，那么她也是父母之一。

(2) Brother 兄弟关系的关键条件有三个：

1. 两个人有共同父母；
2. “兄弟”这个角色本身必须是男性；
3. 不能把一个人和自己判成兄弟。

一个较自然的写法是：

```
(defrule infer-brother
  (parent-of (parent ?p) (child ?x))
  (parent-of (parent ?p) (child ?y))
  (male (person ?x))
  (test (neq ?x ?y))
  =>
  (assert (brother-of (brother ?x) (person ?y))))
```

这里的意思是：

- 如果 ?x 和 ?y 共享同一个父母 ?p；
- 并且 ?x 是男性；
- 并且 ?x 和 ?y 不是同一个人；

那么就可以推出 ?x 是 ?y 的兄弟。

如果想写得更稳，也可以不用 parent-of，直接分别用 father-of 和 mother-of 去推，但既然题目已经给了 parent-of 模板，先推出 parent-of 再推出 brother-of 会更清晰。

第 4 题 提高 CLIPS 规则效率

题目翻译

4. (10 分) 将下面的规则改写得更高效。

```
(defrule three-distinct-points
  ?point-1 <- (point (x ?x1) (y ?y1) (z ?z1))
  ?point-2 <- (point (x ?x2) (y ?y2) (z ?z2))
  ?point-3 <- (point (x ?x3) (y ?y3) (z ?z3))
  (test (and (or (eq ?x1 25)
                  (eq ?x1 56))
              (neq ?point-1 ?point-2)
              (neq ?point-2 ?point-3)
              (neq ?point-1 ?point-3)
              (neq ?x3 89)))
  =>
)
```

对应知识点

这道题对应第八章后半部分关于**模式顺序、测试条件、匹配效率**的内容。重点不在逻辑是否成立，而在于：怎样写得让匹配网络更高效。

解析

原规则的问题主要有两类。

第一，三个 point 模式都非常宽松，系统会先枚举大量点组合，最后才在 test 里统一筛掉大量无效情况。这会让部分匹配数量快速膨胀。

第二，很多本来可以提前写进模式约束里的条件，被拖到了最后一个大 test 中。例如：

- ?x1 必须等于 25 或 56；
- ?x3 不能等于 89。

这些条件若能在模式匹配阶段就过滤掉，会比等三个点全匹配完再 test 更省。

因此可以改写成类似下面的形式：

```
(defrule three-distinct-points-fast
  ?point-1 <- (point (x ?x1&25|56) (y ?y1) (z ?z1))
  ?point-2 <- (point (x ?x2) (y ?y2) (z ?z2))
  (test (neq ?point-1 ?point-2))
  ?point-3 <- (point (x ?x3&~89) (y ?y3) (z ?z3))
  (test (neq ?point-2 ?point-3))
  (test (neq ?point-1 ?point-3))
  =>
)
```

这个版本更高效的原因是：

1. ?x1 的取值约束被提前写进了第一个模式；
2. ?x3 \neq 89 也被提前写进了第三个模式；
3. 不同点之间的不等关系被拆开，尽量在匹配过程中更早筛掉非法组合。

若再进一步强调原则，那么这道题的标准思想可以概括成三句：

1. 越具体的模式越要尽量靠前；
2. 能写进模式约束的条件，不要拖到最后统一 test；
3. 无效候选要尽可能早剪枝。

第 5 题 归结证明

题目翻译

5. (10 分) 使用归结法解决下面的问题。

- 1) 所有爱所有花的人，都被某个人爱着。

2) 任何砸坏一朵花的人, 不被任何人爱。

3) Rockery 爱所有花。

4) 玫瑰是一种花。

5) Rockery 或 Tommy 中有一个人砸坏了那朵名叫 Derrick 的玫瑰。

问题: Tommy 是否砸坏了 Derrick?

对应知识点

这道题对应第七章中的**自然语言形式化、子句化、归结反驳**。它和你之前重点练过的逻辑大题属于同一类。

解析

先设谓词:

- $\text{Love}(x, y)$: x 爱 y
- $\text{Flower}(y)$: y 是花
- $\text{Smash}(x, y)$: x 砸坏了 y

并把 Derrick 看成某一朵具体的玫瑰。

第一步: 形式化

前提 1 可写为:

$$\forall x [(\forall y (\text{Flower}(y) \rightarrow \text{Love}(x, y))) \rightarrow \exists z \text{Love}(z, x)]$$

前提 2 可写为:

$$\forall x \forall y [(\text{Smash}(x, y) \wedge \text{Flower}(y)) \rightarrow \forall z \neg \text{Love}(z, x)]$$

前提 3:

$$\forall y (\text{Flower}(y) \rightarrow \text{Love}(\text{Rockery}, y))$$

前提 4:

$$\text{Flower}(\text{Derrick})$$

前提 5:

$$\text{Smash}(\text{Rockery}, \text{Derrick}) \vee \text{Smash}(\text{Tommy}, \text{Derrick})$$

第二步：先推出 Rockery 被某人爱

由前提 3 可知, Rockery 爱所有花。再结合前提 1, 得到:

$$\exists z \text{ Love}(z, \text{Rockery})$$

也就是说, Rockery 被某个人爱着。

第三步：证明 Rockery 不可能砸坏 Derrick

若假设

$$\text{Smash}(\text{Rockery}, \text{Derrick})$$

由于前提 4 有

$$\text{Flower}(\text{Derrick})$$

代入前提 2 就得到:

$$\forall z \neg \text{Love}(z, \text{Rockery})$$

这表示**没有任何人爱 Rockery**。

但前一步已经推出存在某个 z 使得

$$\text{Love}(z, \text{Rockery})$$

两者矛盾。因此

$$\neg \text{Smash}(\text{Rockery}, \text{Derrick})$$

第四步：由析取前提推出 Tommy 砸坏了 Derrick

前提 5 已知:

$$\text{Smash}(\text{Rockery}, \text{Derrick}) \vee \text{Smash}(\text{Tommy}, \text{Derrick})$$

又已推出:

$$\neg \text{Smash}(\text{Rockery}, \text{Derrick})$$

于是归结可得:

$$Smash(Tommy, Derrick)$$

所以答案是：**Tommy 砸坏了 Derrick。**

这道题如果严格写成归结反驳风格，也可以把“Tommy 没有砸坏 Derrick”加入，再和前提一起推出空子句。但就思路而言，上面的链条已经很清楚：

1. Rockery 爱所有花；
2. 所以 Rockery 被某人爱；
3. 砸花的人不被任何人爱；
4. 因而 Rockery 不可能是砸花者；
5. 所以只能是 Tommy。

第 6 题 HMM 最优路径

题目翻译

6. (15 分) 给定一个由盒子和球构成的隐马尔可夫模型 $\lambda = (A, B, \pi)$ ， T 是观测序列 O 的长度。

$$A = \begin{bmatrix} 0.5 & 0.2 & 0.3 \\ 0.4 & 0.4 & 0.2 \\ 0.2 & 0.3 & 0.5 \end{bmatrix}, \quad B = \begin{bmatrix} 0.5 & 0.5 \\ 0.3 & 0.7 \\ 0.6 & 0.4 \end{bmatrix}, \quad \pi = (0.2, 0.4, 0.4)^T$$

盒子状态集合为 $\{box1, box2, box3\}$ ，球的颜色集合为 $\{red, white\}$ 。

令 $T = 3$ ， $O = (red, white, white)$ ，求隐藏状态序列的最优路径。

对应知识点

这道题对应 HMM 与 Viterbi 算法，是非常标准的动态规划手算题。

解析

我们记

$$\delta_t(j) = \max_{q_1, \dots, q_{t-1}} P(q_1, \dots, q_{t-1}, q_t = j, o_1, \dots, o_t \mid \lambda)$$

它表示：到时刻 t 为止、并且当前停在状态 j 时的最优路径概率。

同时用

$$\phi_t(j)$$

记录最佳前驱状态。

第一步：初始化

第一个观测是 red , 因此:

$$\delta_1(1) = \pi_1 b_1(red) = 0.2 \times 0.5 = 0.1$$

$$\delta_1(2) = \pi_2 b_2(red) = 0.4 \times 0.3 = 0.12$$

$$\delta_1(3) = \pi_3 b_3(red) = 0.4 \times 0.6 = 0.24$$

第二步：计算 $t = 2$

第二个观测是 white 。

到状态 1:

$$\delta_2(1) = \max(0.1 \times 0.5, 0.12 \times 0.4, 0.24 \times 0.2) \times 0.5$$

三项分别是:

$$0.05, \quad 0.048, \quad 0.048$$

所以

$$\delta_2(1) = 0.05 \times 0.5 = 0.025, \quad \phi_2(1) = 1$$

到状态 2:

$$\delta_2(2) = \max(0.1 \times 0.2, 0.12 \times 0.4, 0.24 \times 0.3) \times 0.7$$

三项分别是:

$$0.02, \quad 0.048, \quad 0.072$$

所以

$$\delta_2(2) = 0.072 \times 0.7 = 0.0504, \quad \phi_2(2) = 3$$

到状态 3:

$$\delta_2(3) = \max(0.1 \times 0.3, 0.12 \times 0.2, 0.24 \times 0.5) \times 0.4$$

三项分别是：

$$0.03, \quad 0.024, \quad 0.12$$

所以

$$\delta_2(3) = 0.12 \times 0.4 = 0.048, \quad \phi_2(3) = 3$$

第三步：计算 $t = 3$

第三个观测仍然是 white。

到状态 1：

$$\delta_3(1) = \max(0.025 \times 0.5, 0.0504 \times 0.4, 0.048 \times 0.2) \times 0.5$$

三项分别是：

$$0.0125, \quad 0.02016, \quad 0.0096$$

所以

$$\delta_3(1) = 0.02016 \times 0.5 = 0.01008, \quad \phi_3(1) = 2$$

到状态 2：

$$\delta_3(2) = \max(0.025 \times 0.2, 0.0504 \times 0.4, 0.048 \times 0.3) \times 0.7$$

三项分别是：

$$0.005, \quad 0.02016, \quad 0.0144$$

所以

$$\delta_3(2) = 0.02016 \times 0.7 = 0.014112, \quad \phi_3(2) = 2$$

到状态 3：

$$\delta_3(3) = \max(0.025 \times 0.3, 0.0504 \times 0.2, 0.048 \times 0.5) \times 0.4$$

三项分别是：

$$0.0075, \quad 0.01008, \quad 0.024$$

所以

$$\delta_3(3) = 0.024 \times 0.4 = 0.0096, \quad \phi_3(3) = 3$$

第四步：回溯

最后时刻的最大值是

$$\max(0.01008, 0.014112, 0.0096) = 0.014112$$

对应状态 2，因此：

$$q_3^* = 2$$

再往前回溯：

$$q_2^* = \phi_3(2) = 2$$

$$q_1^* = \phi_2(2) = 3$$

所以最优隐藏状态路径是

$$(3, 2, 2)$$

第 8 题 遗传算法求函数最大值

题目翻译

8. (8 分) 请用遗传算法求下面函数的最大值：

$$f(x) = x * \sin(10\pi * x), \quad x \in [-1, 2]$$

只需简要描述你的解法，不需要把每一步都写得特别细。如果需要，可以写一些伪代码。

对应知识点

这道题对应第六章遗传算法。重点不是把极值精确手算出来，而是会不会把一个连续优化问题翻译成遗传算法流程。

解析

对于这种题，最标准的答法是按遗传算法的结构依次写：

1. 编码方式 可以把区间 $[-1, 2]$ 上的一个实数 x 编码成染色体。做法有两种：

1. **二进制编码**：用若干位二进制串表示区间上的一个近似值；
2. **实数编码**：直接把个体表示成一个实数 x 。

如果题目只要求描述思路，直接采用实数编码最清楚：

$$\text{individual} = x, \quad x \in [-1, 2]$$

2. 适应度函数 既然题目要求最大化

$$f(x) = x \sin(10\pi x)$$

那么最自然的适应度函数就是

$$\text{Fitness}(x) = f(x)$$

若担心出现负值影响轮盘赌选择，也可以平移为

$$\text{Fitness}(x) = f(x) - f_{\min} + \varepsilon$$

其中 $\varepsilon > 0$ 是一个很小的正数。

3. 初始化 随机生成一批个体，例如在区间 $[-1, 2]$ 上均匀采样得到初始种群：

$$x^{(1)}, x^{(2)}, \dots, x^{(N)}$$

4. 选择 根据适应度进行选择，例如轮盘赌选择或锦标赛选择。适应度越高的个体，被保留下来或被选中繁殖的概率越大。

5. 交叉 如果采用实数编码，可以用算术交叉。例如父代 x_1, x_2 生成子代：

$$x' = \alpha x_1 + (1 - \alpha)x_2$$

其中 $\alpha \in [0, 1]$ 为随机数。

6. 变异 对子代加入小扰动：

$$x'' = x' + \epsilon$$

其中 ϵ 可以取很小的随机噪声。变异后若超出区间 $[-1, 2]$ ，则做截断或重新映射回区间内。

7. 终止条件 可以设置:

1. 达到最大迭代代数;
2. 连续若干代最优值提升很小;
3. 已找到足够满意的函数值。

一个完整的答题框架

若考试里要写成简洁的设计题, 下面这样的结构就已经很合格:

1. Randomly initialize a population of x in $[-1, 2]$.
2. Evaluate fitness by $f(x) = x \cdot \sin(10 \cdot \pi \cdot x)$.
3. Select individuals according to fitness.
4. Apply crossover to generate offspring.
5. Apply mutation and keep x in $[-1, 2]$.
6. Repeat until the stopping criterion is satisfied.
7. Output the best individual and its function value.

真正的评分点通常在于: 你有没有把**编码、适应度、选择、交叉、变异、终止**这六个环节写全, 并且都和当前问题挂上钩。

第 9 题 神经网络综合题

题目翻译

9. (15 分) 下图给出了一个单隐藏层的简单神经网络结构。输入为

$$x = [x_1, x_2, x_3]^T$$

隐藏层有两个神经元

$$h = (h_1, h_2)$$

输出记为 y , 不使用偏置。

隐藏层和输出层的激活函数都为 ReLU:

$$\sigma(x) = \max(0, x)$$

损失函数定义为

$$L(y, t) = \frac{1}{2}(y - t)^2$$

其中 t 是目标值。输入层权重为 W , 隐藏层到输出层权重为 V 。

(1) 说明如何利用 W, V, σ, x 得到 y 。(2 分)

(2) 设

$$W = \begin{bmatrix} 1 & 0 & -1 \\ 0 & 2 & 1 \end{bmatrix}, \quad V = [0 \quad 1], \quad x = [1, 0, 2]^T, \quad t = 1$$

计算 y 、 $\frac{\partial L}{\partial W}$ 和 $\frac{\partial L}{\partial V}$ 。(5 分)

(3) “Dead Neuron” 指在训练过程中输出始终为 0 的神经元。基于 (2) 的设定，隐藏层中哪个神经元是死神经元？在保持 ReLU 特性的同时，怎样减少或避免死神经元？(4 分)

(4) 有一个分类神经网络模型使用交叉熵作为损失函数。若某个样本标签为 $(0, 1, 0)$ ，它在第 1 个 epoch 的输出为 $(0.1, 0.8, 0.1)$ ，第 2 个 epoch 的输出为 $(0.3, 0.4, 0.3)$ 。请计算这两个 epoch 的损失值，并说明能否把 accuracy 作为目标函数，为什么。(4 分)

对应知识点

这道题同时考到了前向传播、反向传播、ReLU 特性、交叉熵和目标函数理解，属于神经网络部分比较典型的综合题。

解析

(1) 怎样由 W, V, σ, x 得到 y 单隐藏层网络最基本的计算链就是：

先算隐藏层线性部分：

$$z = Wx$$

再经过 ReLU 得到隐藏层输出：

$$h = \sigma(z) = \sigma(Wx)$$

然后算输出层线性部分：

$$u = Vh$$

最后经过 ReLU 得到输出：

$$y = \sigma(u) = \sigma(Vh) = \sigma(V\sigma(Wx))$$

所以总公式就是：

$$y = \sigma(V\sigma(Wx))$$

(2) 计算 y 、 $\frac{\partial L}{\partial W}$ 、 $\frac{\partial L}{\partial V}$ 先算隐藏层线性输入：

$$Wx = \begin{bmatrix} 1 & 0 & -1 \\ 0 & 2 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \\ 2 \end{bmatrix} = \begin{bmatrix} 1 - 2 \\ 0 + 0 + 2 \end{bmatrix} = \begin{bmatrix} -1 \\ 2 \end{bmatrix}$$

经过 ReLU:

$$h = \sigma(Wx) = \begin{bmatrix} 0 \\ 2 \end{bmatrix}$$

接着算输出层:

$$Vh = \begin{bmatrix} 0 & 1 \end{bmatrix} \begin{bmatrix} 0 \\ 2 \end{bmatrix} = 2$$

因此

$$y = \sigma(2) = 2$$

损失为

$$L = \frac{1}{2}(2 - 1)^2 = \frac{1}{2}$$

接下来求梯度。

因为输出层输入为 2, 大于 0, 所以输出层 ReLU 导数为 1。

先求对输出层权重 V 的梯度:

$$\frac{\partial L}{\partial y} = y - t = 1$$

又因为

$$y = Vh$$

所以

$$\frac{\partial L}{\partial V} = (y - t)h^T = 1 \cdot \begin{bmatrix} 0 & 2 \end{bmatrix} = \begin{bmatrix} 0 & 2 \end{bmatrix}$$

再求对隐藏层的梯度。输出层反传到隐藏层得到:

$$\frac{\partial L}{\partial h} = V^T \frac{\partial L}{\partial y} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

隐藏层中:

- 第一个神经元输入是 -1 , ReLU 导数为 0;
- 第二个神经元输入是 2, ReLU 导数为 1。

因此对隐藏层线性输入的梯度为

$$\frac{\partial L}{\partial z} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

所以

$$\frac{\partial L}{\partial W} = \frac{\partial L}{\partial z} \cdot x^T = \begin{bmatrix} 0 \\ 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 2 \end{bmatrix} = \begin{bmatrix} 0 & 0 & 0 \\ 1 & 0 & 2 \end{bmatrix}$$

综上：

$$y = 2$$

$$\frac{\partial L}{\partial V} = [0 \quad 2]$$

$$\frac{\partial L}{\partial W} = \begin{bmatrix} 0 & 0 & 0 \\ 1 & 0 & 2 \end{bmatrix}$$

(3) 死神经元 在 (2) 的设定下，第一个隐藏神经元的输入是

$$z_1 = -1$$

所以其输出为

$$h_1 = \sigma(-1) = 0$$

而且在这点上 ReLU 的梯度也为 0，因此它就是题目想让你识别的“死神经元”。

如何减少或避免死神经元？如果强调“尽量保持 ReLU 的基本特性”，那么可以写：

1. 使用更合适的参数初始化，例如 He 初始化；
2. 学习率不要过大，避免参数一下子把神经元推到长期负半轴；
3. 适当使用正偏置，让神经元更容易落在激活区；
4. 使用 Batch Normalization 等方法稳定输入分布。

如果题目允许提变体，也可以补一句：Leaky ReLU、PReLU 等方法能进一步减轻死神经元问题，但它们已经不是严格意义上的标准 ReLU。

(4) 交叉熵与 accuracy 标签是 one-hot 向量 $(0, 1, 0)$ ，因此交叉熵只需要看第二类对应的预测概率。

第 1 个 epoch 的输出是 $(0.1, 0.8, 0.1)$ ，所以损失为

$$L_1 = -\ln 0.8 = 0.22$$

第 2 个 epoch 的输出是 $(0.3, 0.4, 0.3)$ ，所以损失为

$$L_2 = -\ln 0.4 = 0.92$$

因此，第 2 个 epoch 的损失更大，说明虽然分类结果可能仍然猜对了第二类，但模型对正确类别的置信度下降了。

能不能把 accuracy 当作目标函数？一般**不能直接把 accuracy 作为训练目标函数**。原因有两点：

1. accuracy 是离散的、不可导的，不适合梯度下降；
2. accuracy 只看“分对还是分错”，不关心概率分布细节，而交叉熵能区分“正确但不够自信”和“正确且很自信”。

因此，训练时通常优化交叉熵，而把 accuracy 当作评价指标。

第 10 题 卷积层、池化层与网络设计

题目翻译

10. (10 分) 在一个卷积神经网络中，考虑一个卷积层 C 和一个最大池化层 P 。

- a) 层 C 的输入有 8 个通道，每个通道大小为 16×16 。
- b) 层 C 有 16 个卷积核，每个卷积核大小为 $4 \times 4 \times 8$ 。
- c) 边界填充大小为 $p = 1$ ，步长为 $s = 2$ 。
- d) 层 P 对层 C 的输出特征图做 2×2 最大池化，池化步长为 2。

(1) 卷积层 C 有多少个参数？池化层 P 输出特征图的维度是多少？(3 分)

(2) 假设要分析人脸图像中表达的情绪。输入图像是一个大小为 32×32 的三通道的 RGB 图像。设：

- 情绪分为两类：积极和消极；
- 情绪强度可以取 0 到 1 之间的数。

因此，一个输入图像的输出应包括情绪类别和情绪强度。请基于网络层 C 和 P 设计一个神经网络来完成该任务，并说明网络结构、超参数、损失函数以及设计理由。(7 分)

对应知识点

这道题对应卷积神经网络基础。前半问是计算题，后半问是设计题。

解析

(1) 参数个数与输出维度 先算卷积层参数个数。

每个卷积核大小为

$$4 \times 4 \times 8$$

因此一个卷积核的权重数是

$$4 \times 4 \times 8 = 128$$

若每个卷积核带一个偏置，那么每个卷积核总参数数为

$$128 + 1 = 129$$

一共有 16 个卷积核，所以总参数数为

$$16 \times 129 = 2064$$

如果老师默认不计偏置，也可以写成 2048。但在通常习惯下，卷积层参数数默认包括偏置，因此更常见答案是

$$2064$$

接下来算卷积层输出空间尺寸。公式是

$$\text{out} = \left\lfloor \frac{n + 2p - k}{s} \right\rfloor + 1$$

这里：

- 输入边长 $n = 16$
- 卷积核边长 $k = 4$
- padding $p = 1$
- stride $s = 2$

所以输出边长为

$$\left\lfloor \frac{16 + 2 - 4}{2} \right\rfloor + 1 = \left\lfloor \frac{14}{2} \right\rfloor + 1 = 7 + 1 = 8$$

由于有 16 个卷积核，因此卷积层输出特征图维度为

$$16 \times 8 \times 8$$

然后做 2×2 最大池化，步长为 2，所以边长减半：

$$8 \rightarrow 4$$

因此池化层输出维度为

$$16 \times 4 \times 4$$

(2) 网络设计 这是一道开放题，没有唯一答案，关键是设计得合理。一个比较自然的方案如下。

网络结构 输入:

$$3 \times 32 \times 32$$

第一层卷积:

- 使用 16 个卷积核;
- 卷积核大小可以取 3×3 或 5×5 ;
- 步长 1;
- padding 适当设置为 1 或 2;
- 激活函数使用 ReLU。

然后接一个最大池化层:

- 池化窗口 2×2 ;
- 步长 2。

接着可以再堆一组卷积层和池化层, 例如:

$$\text{Conv} \rightarrow \text{ReLU} \rightarrow \text{Pool} \rightarrow \text{Conv} \rightarrow \text{ReLU} \rightarrow \text{Pool}$$

之后把特征图展平, 接全连接层, 最后分成两个输出头:

1. **分类头**: 输出 2 维, 用 softmax 表示正向/负向情绪类别;
2. **回归头**: 输出 1 维, 用 sigmoid 或直接线性输出再截断到 $[0, 1]$, 表示情绪强度。

损失函数 因为这个任务同时包含分类和回归, 所以总损失可以写成加权和:

$$L = L_{\text{cls}} + \lambda L_{\text{reg}}$$

其中:

- 分类部分用交叉熵损失;
- 强度预测部分用均方误差损失。

例如:

$$L_{\text{cls}} = - \sum_i y_i \log \hat{y}_i$$

$$L_{\text{reg}} = (s - \hat{s})^2$$

这里 s 是真实情绪强度, $\hat{s}$ 是预测强度。

设计理由 这个设计合理的原因在于:

1. 卷积层适合提取局部空间特征, 例如眼睛、嘴角、眉毛等;
2. 池化层能降低维度、减少计算量, 并增强平移鲁棒性;

3. 情绪类别和情绪强度虽然相关，但不是同一个任务，因此分成两个输出头更自然；
4. 采用分类损失加回归损失，能同时优化“是什么情绪”和“情绪有多强”。

如果想把答案写得更像考试中的设计题，可以整理成如下简洁版本：

```
Input: 3x32x32 RGB image
Conv(16, 3x3) + ReLU
MaxPool(2x2)
Conv(32, 3x3) + ReLU
MaxPool(2x2)
Flatten
FC
Branch 1: Softmax for 2-class emotion classification
Branch 2: Sigmoid/Linear for emotion strength regression
Loss = CrossEntropy + lambda * MSE
```

这样的结构已经足够完整，既给出了网络，也给出了损失与理由。